Ignoring some steps we can simplify this expression as:

$$\begin{cases} (-1)^{b_0} |+\rangle & \text{if } b_0 = b_1 \\ (-1)^{b_0} |-\rangle & \text{if } b_0 \neq b_1 \end{cases}.$$

After the oracle, if $b_0 = b_1$ we obtain the state $|+\rangle$ up to a global phase; if $b_0 \neq b_1$ we obtain the state $|-\rangle$ up to a global phase. This phase does not modify the position of the *qubit* on the Bloch sphere, and we can already distinguish the two states with a measurement parallel to the x axis. Since, conventionally, measurements are performed with respect to the z axis, we apply the H gate once more obtaining

$$\begin{cases} (-1)^{b_0} |0\rangle & \text{if } b_0 = b_1 \\ (-1)^{b_0} |1\rangle & \text{if } b_0 \neq b_1 \end{cases}.$$

The act of measurement discards the global phase and allows us to distinguish the two cases with certainty: we obtain $|0\rangle$ if the unknown bits were equal (even parity), $|1\rangle$ if the unknown bits were different (odd parity). It is interesting to note that throughout the entire algorithm we never learn the value of the individual bits, but only whether they are equal or not.

Deutsch's algorithm allows us to halve the number of oracle queries, and it can be extended to an arbitrary number of bits whose parity we want to determine. This example concludes our discussion of the quantum gate-based paradigm. We will encounter this paradigm again in Chapter 7, where we will combine it with the concepts of quantum annealing that we now illustrate.

2.3 ADIABATIC QUANTUM COMPUTING

Adiabatic Quantum Computing (AQC) is a paradigm of quantum computation alternative to the gate-based quantum model. The two models are computationally equivalent, but deeply different in the nature of computation. AQC has an analog nature that contrasts with the digital one of the gate-based paradigm. When we adopt AQC, we study the evolution of a physical system over time.

Before describing this approach to quantum computation in more detail, it is necessary to give some definitions that will be useful in the following discussion. We call a *quantum dynamical system* a system formed by n *qubits* that evolves over time according to certain forces. The time evolution is determined by the Hamiltonian $\mathbf{H}(t)$[17] of the system and described by the Schrödinger equation (Equation 1.15). As we observed in Section 1.4.3, $\mathbf{H}(t)$ is a Hermitian operator and the observable associated with it is the energy of the system. The eigenvalues of $\mathbf{H}(t)$ at a given instant represent all the possible energy levels of the system that we can measure. We call this set of values the *energy spectrum* of the system. We are interested in two particular energy levels: the lowest energy one, called the *ground state*, and the *first excited state*, the energy level immediately above the ground state.

17 We write it in this way to emphasize that in general $\mathbf{H}$ also depends on time.

2.3.1 AQC algorithm

AQC algorithms are designed to solve optimization problems formulated as follows: Given an objective function $f : \mathbb{D}^n \to \mathbb{R}$ defined on n variables $x = x_1, \ldots x_n$ from some discrete domain $\mathbb{D}$, find an assignment of values to x that minimizes $f(x)$.

An AQC algorithm to solve optimization problem P with objective function $f(x)$ is described by a time-varying Hamiltonian $\mathbf{H}(t)$ that defines the evolution of the system Q composed of n *qubits*. $\mathbf{H}(t)$ is specified by three components:

- an *initial Hamiltonian* $\mathbf{H}_i$ chosen so that its ground state is easy to find;
- a *final* or *problem Hamiltonian* $\mathbf{H}_f$ that encodes the objective function so that the ground state of the system is an eigenstate of $\mathbf{H}_f$ having minimum eigenvalue. That is, the ground state of the system corresponds to an optimal solution to P;
- an *adiabatic evolution path*, a function $s(t)$ that decreases from 1 to 0 as time goes from 0 to t_f

The AQC algorithm is defined by $\mathbf{H}(t)$ which creates a gradual (adiabatic) transition from $\mathbf{H}_i$ to $\mathbf{H}_f$ according to

$$\mathbf{H}(t) = s(t) \cdot \mathbf{H}_i + (1 - s(t)) \cdot \mathbf{H}_f \tag{2.5}$$

To create the initial Hamiltonian we apply to all *qubits* the Pauli operator X (σ_x in Equation 1.10) —this operator describes the observable spin parallel to the x axis, consequently— the associated eigenvectors are $|+\rangle$ and $|-\rangle$. In particular, if $\mathbf{H}_i$ is defined as

$$\mathbf{H}_i = \sum_{i=1}^{n} \frac{1}{2}(1 - X\, q_i)$$

where $X\, q_i$ indicates the application of the Pauli gate X to the *qubit* i; then the ground state is formed by all n *qubits* in the state $|+\rangle$. This means that a state has been created in which the eigenstates in the standard basis ($\{\, |0\rangle\, , |1\rangle\, \}$) are equiprobably observable.

In general $\mathbf{H}_i$ is specified in a basis orthogonal to that of the final Hamiltonian in order to create a state of maximum superposition.

Starting from the known ground state of $\mathbf{H}_i$ and evolving thanks to $s(t)$ up to $\mathbf{H}_f$, the *adiabatic theorem* ensures that, under certain conditions, Q will always remain in the ground state of $\mathbf{H}(t)$ and at time t_s we will be able to read in the configuration of the *qubits* the solution of problem P.

At first glance it might seem that this formulation limits the capabilities of AQC; in reality, we can naturally rewrite many problems in **NP** as optimization problems. In Section 2.1.3 we mentioned the problem of prime factorization as a candidate to demonstrate the superiority of quantum computing. This problem can be translated into an optimization problem in which, given an integer N, one must find two integers $x, y > 1$ such that they minimize $f(x, y) = (N - x \cdot y)^2$.

2.3.2 Theoretical results

Let us see some results on which AQC is based, which define the classes of problems that can be addressed and establish the relationship between AQC and the gate-based model.

Adiabatic theorem

This theorem guarantees, under fairly strict conditions, that the computation carried out by QAC actually ends in the ground state of $\mathbf{H}_f$. Let us consider the algorithm $\mathbf{H}(t)$ that evolves over time $t : 0 \to t_f$; we remap the flow of time with $s(t)$ strictly decreasing from 1 to 0 while $t : 0 \to t_f$. The Hamiltonian associated with $s(t)$ is $\tilde{\mathbf{H}}(s)$. Finally, we call $\tau(s)$ the rate at which $\tilde{\mathbf{H}}(s)$ evolves as a function of $s(t)$. Denoting by $|\Phi_s\rangle$ the state of the system as a function of $s(t)$, we can rewrite the Schrödinger equation 1.15 for our system as:

$$\frac{\partial}{\partial s}|\Phi_s\rangle = -i\tau(s)\tilde{\mathbf{H}}(s)|\Phi_s\rangle .$$

Assuming that throughout the entire evolution of the system the ground state is always unique, we call $\delta(s)$ the *spectral gap*: the difference between the eigenvalues of the ground state and of the first excited state, at any time s. We call δ_m the *minimum spectral gap* $\delta_m = \min_s \delta(s)$. We assume that

$$\tau(s) \ll \frac{\left|\left|\frac{\partial}{\partial s}\tilde{\mathbf{H}}(s)\right|\right|}{\delta_m^2} \tag{2.6}$$

where $||\cdot||$ is the matrix norm induced by the L_2 metric.

Denoting by $|\Phi_g\rangle$ the ground state of the final Hamiltonian, the *adiabatic theorem* states that:

Theorem 9.

- *If the system is in the ground state at $s = 0$, and*
- *if δ_s is strictly greater than 0 throughout time s, and*
- *if the process evolves slowly enough to obey Equation 2.6*

then the process will finish in the ground state $|\Phi_{t_f}\rangle = |\Phi_g\rangle$ with sufficiently high probability.

Temporal complexity

To obtain an upper bound on the computational time t_f for a problem P of size n, it is necessary to find an upper bound for the numerator $\left|\left|\frac{\partial}{\partial s}\tilde{\mathbf{H}}(s)\right|\right|$ and a lower bound for the denominator δ_m^2 in Equation 2.6.

The numerator is bounded by the maximum difference between the eigenvalues obtainable from the Hamiltonians $\mathbf{H}_i$ and $\mathbf{H}_f$. It is possible to show that this difference is polynomial in the size n of the problem.

In general, the analysis of the minimum spectral gap δ_m^2 for a given algorithm $\mathbf{H}(s)$ and a given instance I of a problem P is difficult. In particular, when trying to develop an algorithm $\mathbf{H}_L(s)$ to decide a language $L \in \mathsf{NP}$, it is not known whether δ_m^2 is polynomial or exponential for $\mathbf{H}_L(s)$. Establishing this would mean proving $\mathbf{P} \neq \mathsf{NP}$ or $\mathbf{P} = \mathsf{NP}$ in this computational model.

Relation with gate based model

By studying the relationships between AQC and quantum gate computing, it has been shown that a universal version of AQC can simulate the gate-based model with at most polynomial slowdown. It has also been shown that the converse is true: the quantum gate model can simulate any AQC computation efficiently. These two theorems show that adiabatic quantum computing and quantum gate computing are polynomially equivalent computational models. The class of problems solved efficiently by AQC algorithms is therefore **BQP**.

2.3.3 Quantum annealing

Quantum annealing (QA) is an approach based on heuristics that allows solving combinatorial optimization problems. It is a system inspired by *simulated annealing*, which in turn is inspired by thermodynamics to propose a stochastic algorithm for finding the minimum energy of a system.

There is a close relationship between AQC and QA; in particular, it is established that an algorithm for quantum annealing can be implemented natively with the instruction set provided by AQC. The two approaches are so closely connected that the two terms are often used interchangeably, and in this work we can also consider them as synonyms. However, there are differences, not so much in the algorithm, but in the reference computational model:

- An AQC algorithm is designed to be implemented in the universal AQC model. This is the computational model discussed in the previous section and is polynomially equivalent to the quantum gate-based model;
- A QA algorithm aims to solve a certain type of problems: combinatorial optimization problems. This allows imposing restrictions on the final Hamiltonian and relaxing the conditions of the AQC model.

In a QA algorithm, the final Hamiltonian $\mathbf{H}_f$ represents a classical ground state, that is, without superposition at the time of reading the results. The condition that is relaxed with respect to AQC, instead, is the adiabaticity of the evolution of the physical system. In particular, it is no longer required that the entire computation occurs in the ground state of $\mathbf{H}(t)$.

Annealing

Let us now examine the ideas underlying annealing-based systems, starting from simulated annealing developed for classical computation models, and then seeing how quantum annealing fits into the discussion and what advantages the latter has over the classical model.

SIMULATED ANNEALING: This stochastic approach allows finding the minimum of a multivariate function. The method is inspired by thermodynamics and aims to find the most stable configuration of the system, that is, the one with the lowest energy. The function to be minimized therefore represents the energy of the system and is generally composed of many local minima separated by energy barriers. To overcome these barriers, energy is supplied to the system. In simulated annealing this energy is thermal energy.

The physical system starts in a certain (non-optimal) configuration and with a certain (simulated) temperature T. The system changes configuration according to a certain heuristic and in the new configuration it will have a new energy; let ΔE be the difference between the energy in the second configuration and the energy in the first. If $\Delta E < 0$ the new configuration is accepted. If $\Delta E > 0$ the transition is not immediately discarded but is accepted with a certain probability equal to the Boltzmann factor $e^{-\frac{\Delta E}{k_b T}}$ where k_b is the Boltzmann constant. We observe that the higher the temperature, the higher the probability of overcoming high energy barriers. As the temperature decreases —thus assuming we are approaching the global minimum— transitions to higher-energy configurations become rarer. When we reach $T = 0$ the total energy is given only by the internal energy of the system, and, at the end of the annealing, the system is in the state of minimum internal energy.

The time τ required to evolve from $T = T_i$ to $T = 0$ is called the *thermal relaxation time of the system*. If the system is ergodic, the relaxation time increases linearly with the size of the problem (while the possible configurations increase exponentially); in this case simulated annealing is an efficient algorithm for finding the ground state of the system. However, if the system is not ergodic, any thermal fluctuation at finite temperature would require an infinite relaxation time to bring the system to the minimum energy configuration; that is, if the system is not ergodic $\tau \to \infty$.

QUANTUM ANNEALING: Quantum annealing, like its classical version, can be seen as a heuristic that moves within a solution space searching for the best one. The way the space is explored by a QA algorithm is described by a parameterized Markov chain. To move from one configuration to another, temperature T is no longer used but a *transverse field coefficient* Γ (also called the *tunneling coefficient*). Like T, Γ also starts with a high value and decreases to zero following a certain scheduling.

Quantum annealing introduces a *transverse field* or *disordering Hamiltonian* $\mathbf{H}_d$ that does not commute with $\mathbf{H}_f$ and is scaled with the tunneling coefficient. The Hamiltonian that describes the time evolution of the system is therefore:

$$\mathbf{H}(t) = \mathbf{H}_f + \Gamma(t)\mathbf{H}_d \tag{2.7}$$

The disordering Hamiltonian $\mathbf{H}_d$ introduces kinetic energy to the annealing process in the form of quantum fluctuations of the solution space. Decreasing $\Gamma(t)$ moves the system closer to $\mathbf{H}_f$ while damping the quantum fluctuations. Figure 2.10 graphically shows the advantage of QA compared to the classical counterpart: high values of $\Gamma(t)$ allow the computation to escape local minima by *tunneling* through hills instead of climbing over them incrementally. This allows the algorithm to move faster and further across the landscape early in the process.

Another advantage of QA is that during the evolution the system is not in a single state, but in a superposition of states whose probability is defined by $\mathbf{H}(t)$. This idea is also adopted by algorithms that implement QA on classical architectures. In this case we refer to *simulated quantum annealing* and the

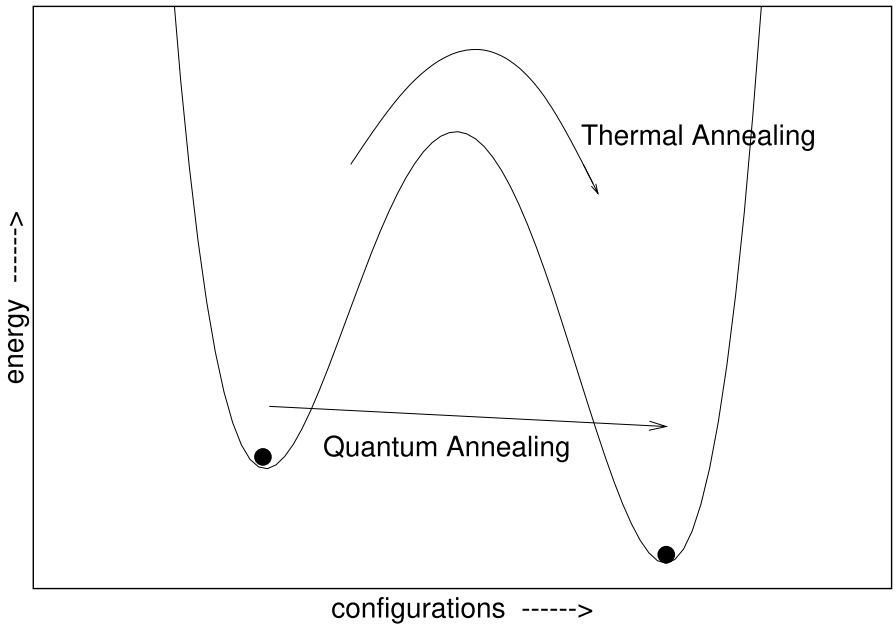

Figure 2.10: Simulated and quantum annealing [**bibid**]

objective is to try to capture the superposition property by representing many states in parallel[18].

Formulating a problem for annealing

Let us see how to formulate a problem so that it can be solved with a *quantum annealer* (a machine capable of implementing QA). We will obtain two equivalent formulations that describe problems solvable with quantum, simulated, or simulated quantum annealing. For now we focus on the theoretical description of the formulation. For a practical implementation compatible with a QA architecture, see Section 4.3.

ISING MODEL: it is a mathematical model that describes phase transitions and other properties of a physical system that evolves over time. The problem is defined by n particles arranged on a graph $G = (V, E)$ that can be described as a d-dimensional grid. Each particle can be in a spin state $\in \{+1, -1\}$. We call *spin configuration* $s = s_1, \ldots, s_n$ an assignment of spins to the particles.

Two types of forces can act on the described system:

- *external forces* h_i applied to individual particles;
- *interaction forces* J_{ij} between grid neighbors (two vertices v_i and v_j connected by an edge e_{ij}). If there are no edges between v_i and v_j we assume $J_{ij} = 0$.

The energy of a given configuration is defined by

$$\mathbf{H}(s) = \sum_i h_i \cdot s_i + \sum_{j<i} J_{ij} \cdot s_i s_j \tag{2.8}$$

18 A more in-depth discussion on simulated quantum annealing is beyond the scope of this work, but can be found in [**bibid**].

Let us see how to encode this energy function in a quantum annealer. Considering a collection of n *qubits* $Q = q_1, \ldots, q_n$ arranged in the grid defined by the graph $G = (V, E)$, the state of Q is $|\Phi\rangle = |\phi_1, \ldots, \phi_n\rangle$. We can translate this energy into the problem Hamiltonian H_f:

$$\mathbf{H}_f = \sum_{i \in V} h_i \cdot Z\, q_i + \sum_{(i,j) \in E} J_{ij} \cdot Z\, q_i\ Z\, q_j. \tag{2.9}$$

Where $Z\, q_i$ indicates the application of the Pauli gate Z to the *qubit* i. This is a valid Hamiltonian for a QA algorithm and a quantum annealer is able to retrieve the solution at minimum energy.

QUBO PROBLEM: The Ising Model, defined on spins $s_1 \in \{+1. - 1\}$. An equivalent formulation, but closer to the usual one in computer science, is the *Quadratic Unconstrained Binary Optimization* problem (QUBO form). In this formulation we consider n Boolean variables $x_i \in \{0, 1\}$ and, considering an upper triangular matrix Q_{ij} of weights of dimension $n \times n$, we want to minimize the function

$$f_{QUBO}(x) = \sum_{i,j} Q_{ij} \cdot x_i x_j. \tag{2.10}$$

It is possible to show that a problem in QUBO form always has a corresponding version in Ising model form, and to move from one formulation to the other one imposes $s_i = 1 - 2x_i$ [**bibid**]. It is also possible to show that the ground state of the problem in QUBO form is the same as that of the problem in Ising form up to a finite offset δ_e

$$\min_{s \in \{-1,+1\}} \sum_i h_i \cdot s_i + \sum_{j<i} J_{ij} \cdot s_i s_j = \min_{x \in \{0,1\}} \sum_{i,j} Q_{ij} \cdot x_i x_j + \delta_e. \tag{2.11}$$

2.4 CONCLUSION

In this work we have justified the interest in quantum computing by showing how this new paradigm is positioned within the study of computational complexity and in the definition of classes of computational problems.

We started by defining theoretical models that describe classical computation such as the Turing machine and the circuit model. Starting from these, we classified computational problems based on the resources required to solve them.

We then introduced quantum computation theoretically (through quantum circuits) and showed how the classes of computational problems change when considering this new paradigm. In particular, we observed that proving the computational advantage of the quantum paradigm would lead to significant results also in classical computation, making it possible to prove that $\mathsf{P} \neq \mathsf{PSACE}$.

The chapter then continues by defining the quantum computation paradigm based on gates, this time in a less theoretical way, observing how a *qubit* is defined in a computational model and what it means to apply a quantum gate to a system of *qubits* to modify its state. The section on quantum gates concludes with the Deutsch algorithm, which shows a first speedup (even if only polynomial) of quantum computers compared to classical architectures.

We concluded the chapter by examining another approach to quantum computation, this time an analog approach: AQC. We described what an AQC algorithm

consists of and which conditions must be satisfied for this algorithm to allow us to obtain the solution to our problem. Finally, we moved from the universal AQC model to QA algorithms, losing generality but relaxing the constraints imposed on computation. We derived two forms for formulating a problem suitable for QA (QUBO form and Ising form) and highlighted their equivalence.

These two formulations that we have obtained are precisely the objective of the rest of our work: transforming structured knowledge and a query on this knowledge into a formulation suitable for QA.